



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2023/2024

Modelação e Simulação de Sistemas Naturais
Relatório: Trabalho Prático 3

Turma: 31D

Grupo: 30

Nome: Miguel Alcobia

Número: 50746

Nome: Fábio Pestana

Número: 50756

Professor: Paulo Vieira

Data: 21 de Dezembro de 2023

Índice

Lista de Figuras	3
Introdução	4
A - Função Logística	5
3 - Efeito Borboleta	5
B - Jogo Caos	6
1 - Versão Simples	6
2 - Versão Número Arbitrário de Pontos Fixos	7
C - Gramáticas de Lindenmayer	9
1 - Objetos por L-Systems	10
2 - Árvores de Fruto	11
D - Conjunto de Julia e Mandelbrot	12
1 - Gradiente do conjunto de Mandelbrot	12
Bibliografia	13

Lista de Figuras

Figura 1 - Gráfico do efeito borboleta.....	6
Figura 2 - Resultado da implementação mais simples do Jogo do Caos.....	7
Figura 3 - Resultado com 3 pontos fixos.....	8
Figura 4 - Resultado com 3 pontos fixos.....	8
Figura 5 - Resultado com 7 pontos fixos.....	9
Figura 6 - LSystemApp.....	10
Figura 7 - ForestWithFruitApp.....	11
Figura 8 - MendelbrotApp.....	12

Introdução

Neste trabalho, os alunos tinham que estudar o funcionamento e o comportamento da função logística; implementar pelo menos uma versão do Jogo do Caos; gramáticas de Lindenmayer e conjuntos de Julia de Mandelbrot.

Além disto, os alunos ainda podiam explorar ferramentas para a criação de fractais, elaborar um ensaio sobre fractais e construir uma floresta animada

Todas as implementações foram feitas com Processing 3 em Java, tirando um pequeno código para o estudo do efeito borboleta.

A - Função Logística

3 - Efeito Borboleta

O efeito borboleta é frequentemente associado à Teoria do Caos e refere-se à ideia de que pequenas mudanças nas condições iniciais de um sistema podem levar a grandes diferenças nos resultados ao longo do tempo.

O efeito borboleta é muitas vezes exemplificado pela ideia de que o bater das asas de uma borboleta poderia desencadear um furacão no outro lado do mundo.

Desenvolveu-se estas funções em Python para demonstrar isso:

```
import matplotlib.pyplot as plt

def func_logistica(r, x0, nIter):
    """função logistica

    Args:
        r : parametro r
        x0 : valor inicial
        nIter : numero de Iterações

    Returns:
        y : Valor de y
    """

    x = x0
    y = [0] * nIter
    for i in range(len(y)) :
        y[i] = r * x * (1-x)
        x = y[i]
    return y

def plot_func_logistica(r, x0, nIter):
    """plot_func_logistica - Desenha o plot da expressão da função logistica

    Args:
        r : parametro r
        x0 : valor inicial
        nIter : numero de Iterações
    """

    y = func_logistica(r, x0, nIter)

    plt.figure()
    plt.plot(y, label=f'r = {r}')
    plt.title(f'Função Logística - r={r}')
    plt.xlabel('Iterações')
    plt.ylabel('Valores de x')
    plt.legend()
    plt.show()
```

Fez-se testes com o $r = 3.9$ e o valor inicial x_0 a 0.8 e a 0.80000001 e o resultado foi o seguinte.

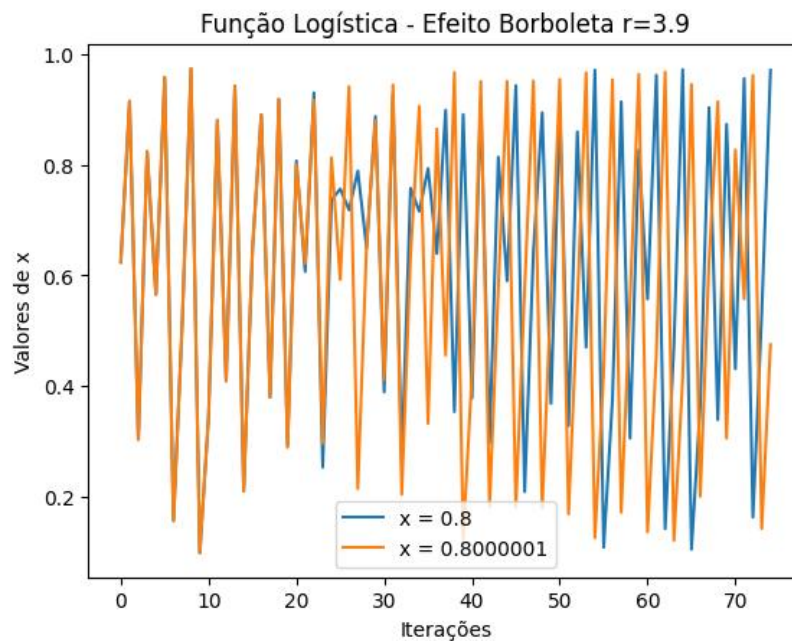


Figura 1 - Gráfico do efeito borboleta

Como mostrado acima, quando o r é 3.9, o simples facto de mudar o valor inicial x_0 de 0.8 para 0.8000001 refletiu-se no desenho de um gráfico completamente diferente.

B - Jogo Caos

1 - Versão Simples

Tendo em vista as dimensões da janela, definimos os três pontos fixos necessários: A, B, C. Em seguida, definido um ponto aleatório X. A variável r recebe um valor que corresponderá a um dos três pontos fixos: 0 corresponde ao ponto A, 1 ao B e 2 ao C.

A função `lerp()` é usada para fazer com que o ponto X passe a ser um ponto intermédio entre ele mesmo e o ponto fixo sorteado em r .

```
if (r == 0){
    p.stroke(255,0,0, 100);
    x = p.lerp(x, ax, 0.5f);
    y = p.lerp(y, ay, 0.5f);
}else if (r == 1){
    p.stroke(0,255,0, 100);
    x = p.lerp(x, bx, 0.5f);
    y = p.lerp(y, by, 0.5f);
}else if (r == 2) {
    p.stroke(0,100,255, 100);
    x = p.lerp(x, cx, 0.5f);
    y = p.lerp(y, cy, 0.5f);
}
```

Este processo foi colocado dentro de um ciclo `for`, que desenha 100 pontos (100 interações) por vez. Consoante o valor de r , a cor dos pontos muda. No final, obtemos um triângulo de Sierpinski com o seguinte aspeto:

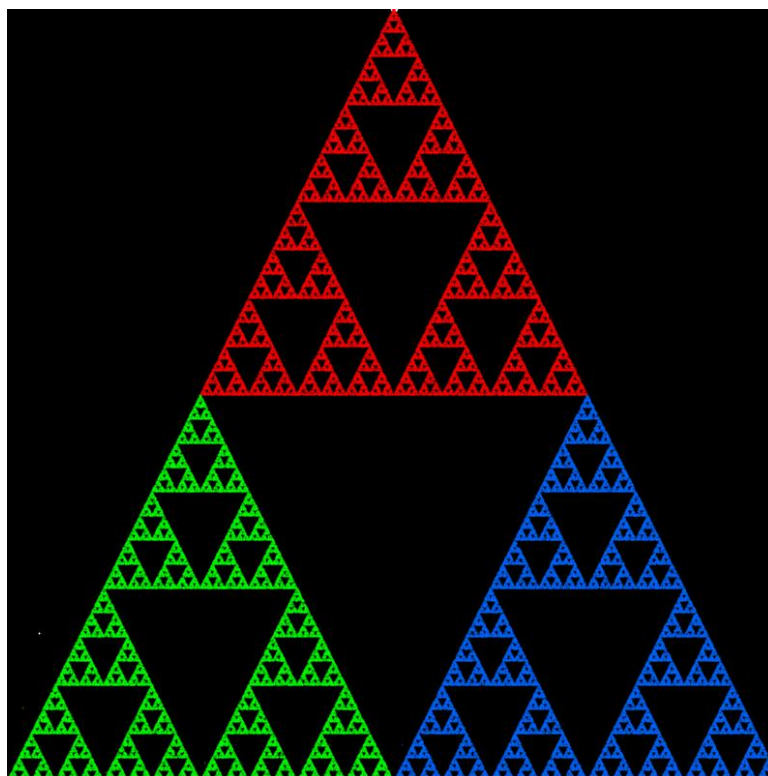


Figura 2 - Resultado da implementação mais simples do Jogo do Caos

2 - Versão Número Arbitrário de Pontos Fixos

Nesta versão, decidiu-se que o número de pontos fixos era definida pelo *user* através dos números do teclado, podendo escolher entre 3 pontos fixos (triângulo) e 9 pontos (eneágono).

Primeiro distribui-se os n pontos de forma uniforme pela janela e fez-se isto como se todos pertencem-se a uma circunferência. Para isto, cria-se um vetor v com a função `fromAngle()`. com o ângulo calculado, depois escala-se e translada-se o vetor para que os pontos fixos fiquem corretamente distribuídos. O vetor v é adicionado a uma lista chamada `points`. Logo depois, cria-se o ponto Atual (`pAtual`) de forma aleatória.

Por último um ciclo `for` percorre essa lista, desenhando cada ponto na janela com o método `point()`.

O método `draw()` verifica se deve dar reset, consoante a contagem de frames. Se não der reset, itera 100 vezes, desenhando o ponto atual e a dinâmica do ponto intermédio foi feita com a ajuda do `lerp()`, tal qual se fez na versão mais simples.

O *user* se clicar em “+” incrementa a variável `resetCount` em 200, o que permite mais iterações antes do reset.

Eis alguns resultados desta implementação:

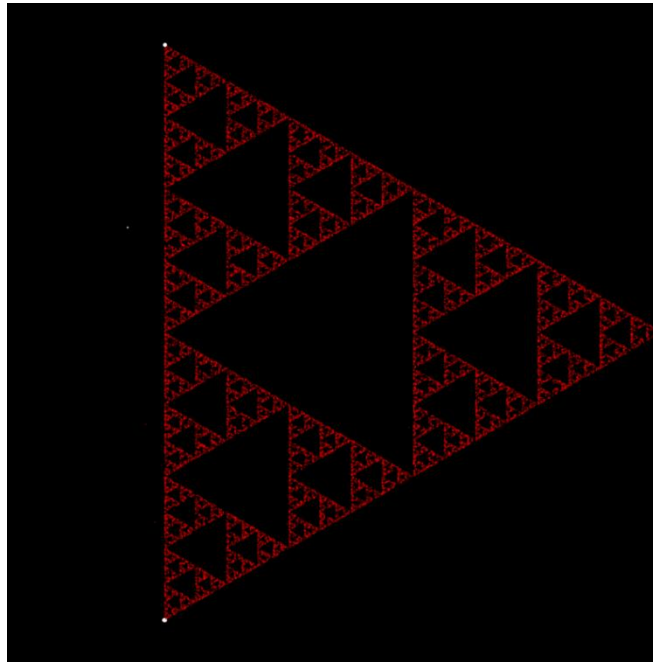


Figura 3 - Resultado com 3 pontos fixos

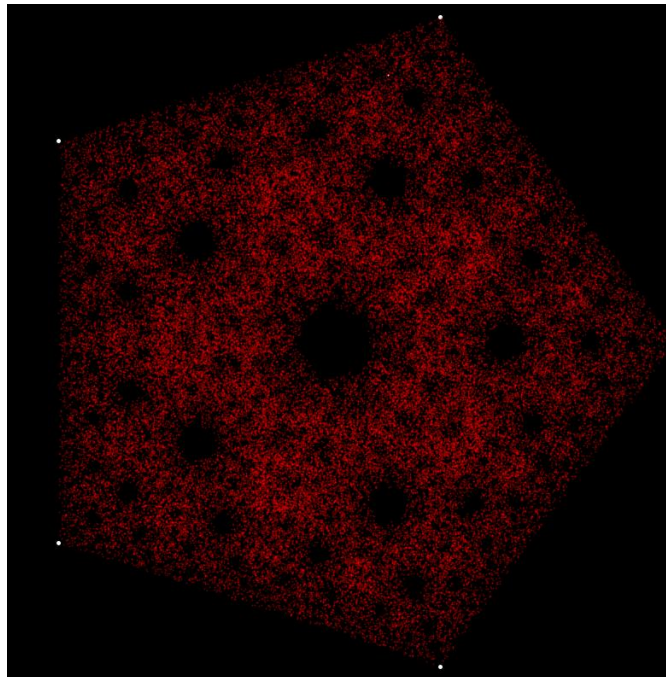


Figura 4 - Resultado com 3 pontos fixos

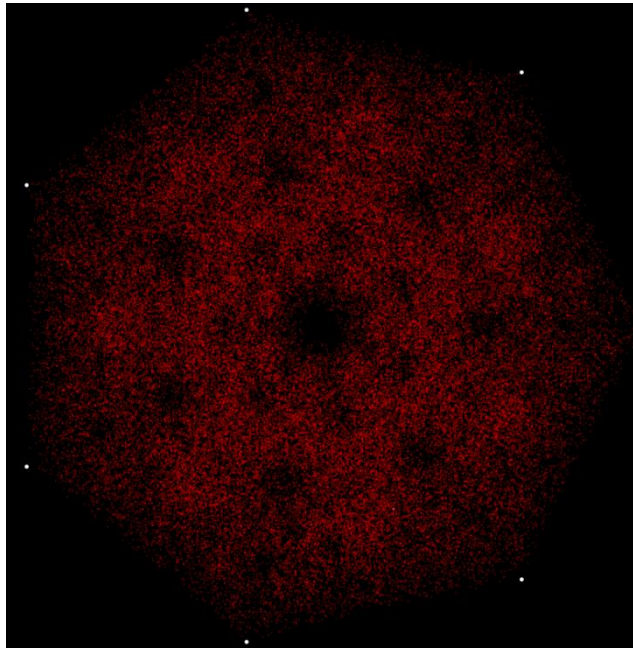


Figura 5 - Resultado com 7 pontos fixos

C - Gramáticas de Lindenmayer

Para este exercício foram desenvolvidas as seguintes classes:

- **Rule:** classe com dois atributos, um char que representa uma letra do alfabeto do sistema (`simbolo`) e uma String que representa a sua regra (`sequencia`).
- **LSystem:** classe que representa um sistema de Lindenmayer e tem 3 atributos, uma String que representa a sequência atual do sistema (`sequence`), um array de Rule's que representa as regras do sistema (`ruleset`) e um int que é inicializado a 0 e conta a geração atual do sistema (`generation`). Esta classe possui o método `nextGeneration()`, que consiste em percorrer a sequência atual e aplicar a cada elemento as regras definidas no `ruleset`, isto resulta na geração de uma nova sequência do sistema e o contador de geração é incrementado.
- **Turtle:** classe que representa uma tartaruga gráfica utilizada para renderizar um sistema de Lindenmayer e tem 6 atributos, um float que representa o comprimento da linha a ser desenhada (`len`) e outro que representa o ângulo de rotação da tartaruga (`angle`), um int que é a cor utilizada para desenhar as folhas e frutas, um boolean que indica se a tartaruga desenha as folhas e frutas (`fruitOrLeave`) e uma lista de floats que representa as probabilidades associadas às folhas e frutas. Esta classe tem um método `render()`, que desenha a sequência do L-system.
- **Tree:** classe que representa uma árvore que cresce ao longo do tempo e tem 11 atributos, um objeto LSystem que representa o sistema de Lindenmayer utilizado para modelar o crescimento da árvore (`ls`), um objeto Turtle responsável por renderizar a estrutura da árvore (`turtle`), um PVector que indica a posição inicial da árvore (`position`), 6 float's, `length` que representa o comprimento da árvore, `growthRate` que representa a taxa de crescimento da árvore, `scalingFactor` o fator de escala aplicado ao comprimento e à taxa de crescimento da árvore, `intervalBetweenSeasons` indica o intervalo de tempo entre cada estação de crescimento, `now` indica o tempo atual em segundos e `nextSeasonTime` indica o tempo da próxima estação de crescimento da árvore, um int que indica o número total de iterações de crescimento da árvore (`numberOfSeasonsToGrow`) e uma instância de PApplet do processing (`p`). Esta classe tem dois métodos importantes sendo estes `grow()` e `display()`. O `grow()` simula o crescimento da árvore ao longo do tempo, atualizando o comprimento da árvore e a `len` da `turtle` com base no tempo decorrido, caso uma estação seja

concluída gera-se uma nova geração do ls e atualiza-se lenght e growthRate através do growthRate. O display() renderiza a árvore numa certa posição segundo o vector position (turtle.setPose()), através do turtle.render().

1 - Objetos por L-Systems

Para esta alínea foi desenvolvida a classe LSystemApp que tem dois objetos fractais, sendo estas as suas respetivas características:

Fractal da esquerda:

Alfabeto: F

Axioma: F

Regras: (F-> FF+[+F-F-F][-F+F+F])

Fractal da direita:

Alfabeto: X, F

Axioma: X

Regras: (X-> F[+X][-X]FX) (F-> FF)

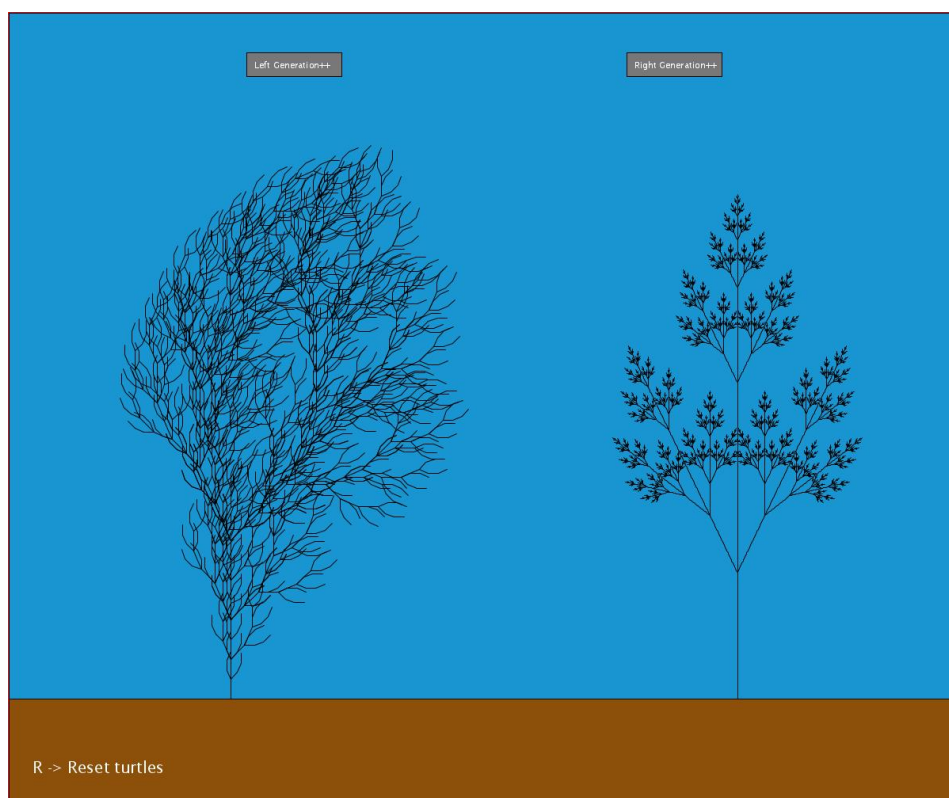


Figura 6 - LSystemApp

Esta classe tem dois botões ao pressionar o da esquerda atualiza-se a geração do fractal da esquerda e no botão da direita acontece a mesma coisa para o fractal da direita. Ao clicar na tecla R é feito um reset dos fractais onde eles voltam para a sua primeira geração.

2 - Árvores de Fruto

Para esta alínea foi desenvolvida a classe ForestWithFruitApp que tem inicialmente 4 objetos Tree, sendo estas as suas respectivas características:

Alfabeto: F, G

Axioma: F

Regras: (F-> G[+F]-F) (G-> GG)

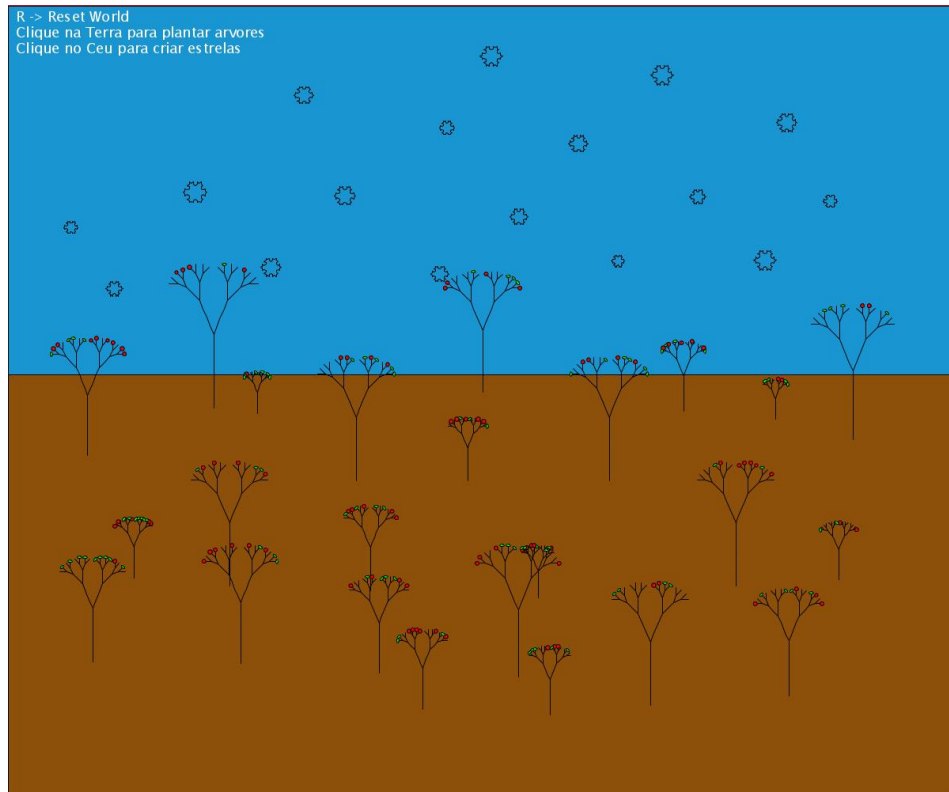


Figura 7 - ForestWithFruitApp

Nesta classe ao clicar no retângulo castanho (a representar a terra) é possível criar novos objetos Tree com as características mencionadas anteriormente, caso clique no retângulo azul (a representar o céu) é possível criar outro tipo de fractais sendo as suas características as seguintes:

Alfabeto: F

Axioma: F++F++F

Regras: (F-> F-F++F-F)

Nesta classe ao clicar na tecla R é feito um reset da app, onde se volta a ter apenas os primeiros 4 objetos iniciais.

D - Conjunto de Julia e Mandelbrot

1 - Gradiente do conjunto de Mandelbrot

A classe desenvolvida para este exercício tem como atributos 5 ints: o número de iterações para atribuir uma cor a esse ponto do conjunto (`numIter`), coordenada inicial em x (`x0`), coordenada inicial em y (`y0`), valor final de x, que define largura da área a ser processada (`dimx`), valor final de y, que define altura da área a ser processada (`dimy`).

No método `display()` da classe, usa-se as funções `loadPixels()` e `updatePixels()` para um melhor tempo de processamento de imagem e percorre-se os pixels todos do ecrã. Com as coordenadas dos pixels, é feita a conversão para como se a janela fosse um sistema de eixos com números complexos, com a ajuda da classe `Complex`. Em seguida, aplica-se a fórmula do conjunto de Mandelbrot e verifica-se se o ponto está fora do conjunto.

Se o pixel estiver dentro do fractal, tem-lhe atribuída a cor preta, caso contrário, a cor do pixel é definida como um valor entre 0 e 255, dependendo do número de iterações com a fórmula $(i \% 16) * 16$. Por exemplo, se o número de iterações for 2, então a cor do pixel será 32. Se o número de iterações for 3, então a cor do pixel será 48. Desta forma, cria-se uma espécie de gradiente, baseado no número de iterações.

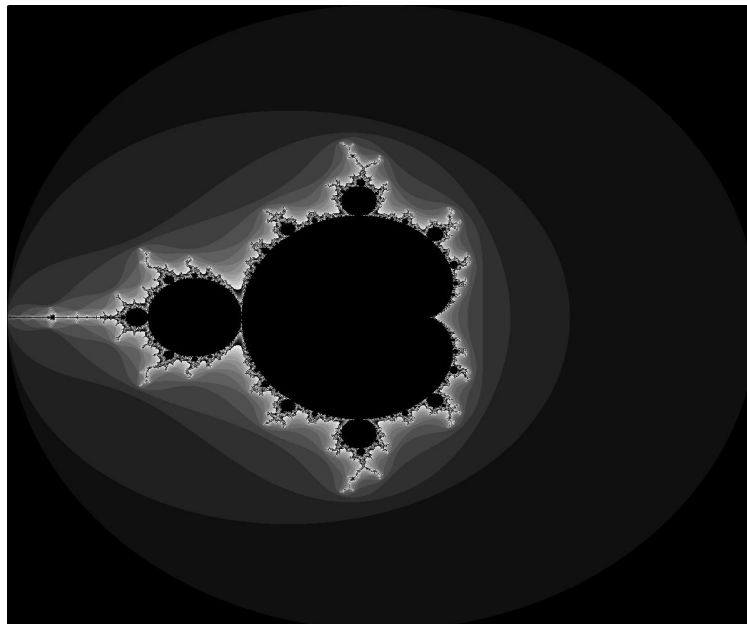


Figura 8 - MendelbrotApp

Bibliografia

Consulta:

Vieira, Paulo. (2023 - 2024). Física e Movimento - Moodle

Vídeos gravados pelo Prof. Arnaldo Abrantes - Moodle